

Subsea — Project-Based Interview Pack

Target: 5-year Full-Stack Developer (Angular · .NET / C# Web API · Docker · Azure · Kendo UI) **Idea:** Every answer is anchored in the **real Subsea project** — so when an interviewer asks "*where did you use X in your actual project?*", you have a concrete feature, pattern, and story. No abstract theory; everything lands back on Schedule-Manager, the Kendo Grid, and the category/subcategory work you actually built.

Why this pack exists: You keep failing on "project understanding." The fix is not more theory — it's being able to take *any* technical question and route it through Subsea. Every answer here ends with an **"In Subsea, I..."** tie-in. Memorize the *flow* of those tie-ins, not the code character-for-character.

How to use this pack

- Each file = one interview area. Read the **Question** → **Answer** → **"In Subsea"** structure.
- The **"In Subsea"** block is your talking point — say it **out loud** until it's natural.
- If an interviewer pushes deeper, walk them through the code snippet — but lead with the *story* (what problem, what you did, what the result was).
- The signature file is `02-kendo-grid-deep-dive.md` — the server-side grid performance story is your strongest differentiator. Know it cold.
- Day before: read `00-ONE-PAGE-CHEATSHEET.md` and `05-behavioral-and-stars.md`.

The sections

#	File	Covers
0	<code>00-ONE-PAGE-CHEATSHEET.md</code>	20 must-know facts + one-liner answers for a 30-min pre-interview glance
1	<code>01-frontend.md</code>	Angular — lifecycle hooks, component communication, RxJS (Observable/Subject/BehaviorSubject), memory leaks + <code>takeUntil</code> / async pipe, restricting duplicate API calls (<code>switchMap</code> / <code>debounceTime</code>), cascading dropdowns (reactive forms), change detection, reusable components
2	<code>02-kendo-grid-deep-dive.md</code>	THE signature file. Kendo UI Grid — client-side vs server-side binding, <code>DataSourceRequest</code> / <code>toDataSourceRequestString</code> (TS + C#), paging/sorting/filtering/grouping, virtual scrolling, large-dataset performance
3	<code>03-backend-dotnet.md</code>	.NET Web API / C# — controllers, <code>[HttpGet]</code> / <code>[HttpPost]</code> , <code>DataSourceRequest</code> on the server, interface vs abstract class, <code>try/catch/finally</code> , global exception handling, dependency injection in .NET
4	<code>04-docker-azure-devops.md</code>	Docker (reproducible builds, "works on my machine" solved, Dockerfile/compose), Azure hosting containers, CI/CD, local/CI/prod environment parity
5	<code>05-behavioral-and-stars.md</code>	60-sec pitch, your role, architecture walkthrough (what to draw), STAR stories (server-side grid debate, production bug methodology, dynamic category management, modularity refactor), "what I'd do differently," Agile/collaboration

Key project facts (know these cold)

- **Subsea** — marine equipment & operations management platform; track and schedule subsea assets and operations. **Apr 2024 – Sep 2024.**
- **Stack:** Angular SPA frontend · .NET (C#) Web API backend · Docker containers · Azure hosting · Kendo UI (Grid).
- **Your role:** Full-stack with an **Angular emphasis**, team of ~5–6. You **owned the Schedule-Manager module front to back** — Angular UI *plus* the .NET endpoints — and you were one of the people who picked up production defects.
- **Architecture:** Angular SPA → REST calls → .NET Web API services → SQL. Containerized with Docker, running on Azure. Schedule-Manager is a feature module with its own routing, services, and Kendo-based grid components.
- **Signature achievement:** moved the **Kendo Grid from client-side to server-side data operations** (paging/sorting/filtering done in the .NET API via `DataSourceRequest`), so large schedule/equipment datasets stayed responsive. You **won a client-vs-server-side debate by demonstrating the lag** with realistic data volume.
- **Dynamic category/subcategory management:** data-driven parent/child hierarchy, cascading dropdowns — choosing a category reactively filters its subcategories. Admins could add/edit categories; the UI reflected it live.
- **Refactor:** pulled repeated grid config and category selectors into **shared, reusable components**; tightened feature-module boundaries.

Honesty note (important — this is what senior interviewers reward)

You are **Angular-emphasis full-stack**, not a deep C# internals guru. That is a *strength* if you frame it honestly:

"I owned Schedule-Manager end to end — the Angular UI and the .NET endpoints behind it. My depth is on the Angular/frontend side; on the .NET side I built the Web API controllers, the `DataSourceRequest` handling, and the data flow. I can tell you exactly how what I built worked, and where I'd bring in a backend specialist for deep runtime/perf tuning."

Interviewers respect *"here's what I built and how it worked, here's the edge of my depth"* far more than someone who bluffs C# garbage-collection internals. When you don't know something: say your understanding, tie it to the closest real Subsea thing you did, and say you'd confirm it. **Never bluff.**

Subsea — One-Page Cheat Sheet · Priyanka R

Stack: Angular SPA · .NET (C#) Web API · Docker · Azure · Kendo UI Grid · SQL
My module: **Schedule-Manager** — owned front-to-back (Angular UI + .NET endpoints). Team ~5–6. Apr–Sep 2024.
Signature win: moved Kendo Grid to **server-side** paging/sort/filter (`DataSourceRequest`) → large datasets stayed fast. Won the client-vs-server debate by **demoing the lag**.

Request flow: Angular component → feature service (`HttpClient`) → **.NET Web API controller** (`[HttpGet]`) → `ToDataSourceResult(request)` → SQL paging → one page of rows back → Kendo Grid renders. Errors → HTTP interceptor + per-call `catchError` → user-facing toast.

20 facts to know cold

1. **Project:** marine equipment & operations platform — track/schedule subsea assets & operations.
2. **My ownership:** Schedule-Manager module, both sides (Angular + .NET), plus production bug fixes.
3. **Signature:** client-side Kendo Grid got sluggish as rows grew → I pushed paging/sort/filter to the .NET API. Only **one page crosses the wire**.
4. **The Kendo pattern:** TS `toDataSourceRequestString(state)` → querystring → C# `[DataSourceRequest] DataSourceRequest → .ToDataSourceResult(request) → DataSourceResult { Data, Total }`.
5. **Why server-side wins:** client-side loads *all* rows into the browser (memory + slow sort/filter); server-side does it in SQL and returns just the visible page.
6. **Virtual scrolling** + requesting only displayed columns = extra responsiveness on top of server-side ops.
7. **Cascading dropdowns:** category → subcategory, data-driven parent/child. On category change I reactively reload dependent subcategories, not the whole tree.
8. **Reactive forms** (`FormBuilder` / `FormGroup`) for the cascade — I subscribe to the category control's `valueChanges` to drive the subcategory list.
9. **Restrict duplicate API calls:** `debounceTime` + `distinctUntilChanged` + `switchMap` (switchMap **Cancels** the previous in-flight request — latest wins).
10. **Memory leaks:** unclosed subscriptions outlive the component. Fix = `async pipe` first, else `takeUntil(destroy$)` completed in `ngOnDestroy`.
11. **Lifecycle hooks (order):** `ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterViewInit` → `ngOnDestroy`. I init data in `ngOnInit`, clean up in `ngOnDestroy`.
12. **Component comms:** `@Input` (parent→child), `@Output EventEmitter` (child→parent), **shared service** + `BehaviorSubject` (siblings/cross-tree).
13. **Observable vs Subject vs BehaviorSubject:** Observable = cold, per-subscriber; Subject = multicast, no initial value; **BehaviorSubject** = multicast **with a current value** (perfect for state — my dropdown selections).
14. **Route guards:** `CanActivate` gates protected routes; **lazy loading** via `loadChildren` so Schedule-Manager ships as its own chunk.
15. **Global exception handling:** Angular `ErrorHandler` + HTTP interceptor on the frontend; **middleware / exception filter** on the .NET side → uniform JSON error, not a 500 stack trace.

16. **Interface vs abstract class (C#):** interface = pure contract, multiple allowed, no implementation; **abstract class** = shared base with some implemented + some abstract members, single inheritance.
 17. **try/catch/finally :** `try` runs risky code, `catch` handles the exception, `finally` **always runs** (cleanup — close/dispose) whether or not it threw.
 18. **DI in .NET:** register services in `Startup / Program` (`AddScoped / AddSingleton / AddTransient`), inject via constructor — same idea as Angular's `providedIn: 'root'`.
 19. **Docker:** same image on my machine, CI, and Azure → killed "works on my machine." Dockerfile builds the .NET API image; Azure hosts the containers.
 20. **Reusable refactor:** pulled repeated Kendo grid config + category selectors into shared components → other modules reuse instead of copy-paste.
-

One-liner answers (if put on the spot)

- **"Difference: switchMap vs mergeMap?"** switchMap cancels the previous (search/latest-wins — I used it to stop stacked grid/filter calls); mergeMap runs all in parallel; concatMap queues in order.
 - **"How stop double API calls?"** `debounceTime` to wait for the user to stop, `distinctUntilChanged` to skip identical values, `switchMap` to cancel the stale request.
 - **"Reactive vs template forms?"** Reactive = defined in TS, typed, testable, great for dynamic/cross-field logic (my cascade); template = simple, HTML-driven, fine for small forms.
 - **"BehaviorSubject vs Subject?"** BehaviorSubject remembers the last value and replays it to new subscribers — a late subscriber still gets current state. Subject only sees values emitted *after* it subscribes.
 - **"Client vs server-side grid?"** Client = everything in the browser, dies on big data; server = paging/sort/filter in SQL, returns one page. I moved Subsea's grid to server-side and demoed the difference.
 - **"Interface vs abstract class?"** Interface = *can-do* contract, multiple; abstract class = *is-a* base with shared code, single. Interface for a capability, abstract class for a common base.
-

If stuck (say it, don't freeze)

"I haven't gone deep on that specific internal — my understanding is X. The closest thing I actually built in Subsea is [real example]. I'd spike it to confirm."

Pull the question toward the **frontend / Schedule-Manager / Kendo Grid** where you're strong, then bridge back. Honest framing: *"I'm Angular-emphasis full-stack — I owned Schedule-Manager's UI and its .NET endpoints end to end, not a C# runtime specialist."*

HTTP: 200 OK · 201 Created · 400 validation · 401 not-logged-in · 403 forbidden · 404 missing · 409 conflict · 500 server error. **Rules:** tie every answer to Subsea · think out loud · never bluff · speak slower than feels natural.

1. Front-End (Angular) — Interview Q&A tied to Subsea

Stack: Angular SPA talking to .NET Web API endpoints. Feature-module architecture, RxJS-heavy, service-based state, Kendo UI components. My module was **Schedule-Manager** — the schedule/equipment grids, the category/subcategory management, and the forms behind them.

How to use this: every answer ends with an "In Subsea, I..." tie-in. That tie-in is the point — it's what turns a textbook answer into project understanding. Lead with the story, drop into code only if pushed.

Q1. Explain Angular lifecycle hooks. Which did you actually use?

Answer. Lifecycle hooks are methods Angular calls at defined moments in a component's life. The ones that matter, in call order:

1. `ngOnChanges(changes)` — runs when an `@Input` value changes (before `ngOnInit` the first time). Good for reacting to a changed input.
2. `ngOnInit()` — runs once after the first `ngOnChanges`; where I do initialization — fetch data, set up subscriptions, build forms. *Not* the constructor, because inputs aren't bound yet in the constructor.
3. `ngDoCheck()` — custom change detection; rarely needed.
4. `ngAfterViewInit()` — after the component's view (and child views / `@ViewChild`) is initialized; where I touch a child component or DOM element that must exist first.
5. `ngOnDestroy()` — right before Angular destroys the component; where I clean up — complete subjects, unsubscribe — to prevent memory leaks.

In Subsea. In the Schedule-Manager grid component I used `ngOnInit` to build the reactive form for filters and fire the first grid load, and `ngOnDestroy` to complete my `destroy$` subject so the grid's data subscriptions didn't leak when the user navigated away. I used `ngOnChanges` in a couple of reusable child components — e.g. a grid-config child that had to re-render when the parent passed a new dataset config as an `@Input`.

```
export class ScheduleGridComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
    this.buildFilterForm();           // set up reactive form
    this.loadGrid(this.gridState);   // first server-side load
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();       // cleanup — no leaks
  }
}
```

Q2. Parent-child component communication?

Answer. Three mechanisms depending on the relationship:

- `@Input()` — parent → child data binding.
- `@Output()` `EventEmitter` — child → parent events.
- **Shared service with a** `BehaviorSubject` — for siblings or components far apart in the tree that don't have a direct parent/child link.

In Subsea. When I refactored the grid config into a **reusable grid component**, the parent Scheduler Manager page passed the column/config and the current data page in via `@Input()`, and the grid emitted user actions (page change, sort, filter, row edit) back up via `@Output()` so the parent could re-query the .NET API:

```
// reusable grid child
@Input() columns: GridColumn[];
@Input() data: GridDataResult;           // { data, total } from the server
@Output() stateChange = new EventEmitter<State>(); // paging/sort/filter changed

onDataStateChange(state: State): void {
  this.stateChange.emit(state);          // let the parent re-fetch server-side
}
```

```
<app-schedule-grid [columns]="cols" [data]="pagedData"
  (stateChange)="loadGrid($event)">
</app-schedule-grid>
```

For the **category/subcategory selection** shared across parts of the module I used a **service with a** `BehaviorSubject` so any component could read the currently-selected category reactively without prop-drilling it through inputs.

Q3. Observable vs Subject vs BehaviorSubject — what's the difference and where did you use each?

Answer. This is really about *cold vs hot* and *whether there's a current value*:

	Observable	Subject	BehaviorSubject
Multicast?	No — cold, runs per subscriber	Yes — hot, shared	Yes — hot, shared
Current value?	n/a	No	Yes — holds "the latest"
New subscriber gets...	its own execution	only values emitted <i>after</i> subscribing	immediately gets the current value , then future ones

	Observable	Subject	BehaviorSubject
Can I push into it?	No (<code>.next()</code> not available)	Yes (<code>.next()</code>)	Yes (<code>.next()</code>)

- **Observable** — the default for things like `HttpClient` calls: cold, each subscribe triggers the request.
- **Subject** — a multicast event bus; new subscribers only see what's emitted *after* they subscribe. Good for "an event happened."
- **BehaviorSubject** — a Subject that remembers its last value and replays it to any new subscriber. Perfect for **state** because a component that subscribes late still gets the current value.

In Subsea. My grid data came from **Observables** (`HttpClient` calls to the .NET API). For the **category/subcategory state** I used a `BehaviorSubject` in a feature service — it held the currently selected category, and any component that subscribed (even one rendered later) immediately got the current selection so the subcategory dropdown could populate correctly:

```
@Injectable({ providedIn: 'root' })
export class CategoryStateService {
  private selectedCategory$ = new BehaviorSubject<Category | null>(null);

  // expose read-only — consumers can't push into it
  readonly category$ = this.selectedCategory$.asObservable();

  selectCategory(cat: Category): void {
    this.selectedCategory$.next(cat); // update the single source of truth
  }
}
```

Follow-up: "Why BehaviorSubject and not a plain Subject here?" Because a component might subscribe *after* the category was already chosen — with a plain Subject it would see nothing until the *next* change. BehaviorSubject replays the current value, so the UI is always consistent.

Q4. How do you prevent memory leaks in Angular? (the #1 RxJS question)

Answer. Memory leaks in Angular almost always come from **RxJS subscriptions that outlive their component**. If you `subscribe()` and never unsubscribe, the stream keeps a reference to the component so it's never garbage-collected, and the callback keeps firing after the view is gone. I handle it in order of preference:

1. `async pipe` in the template — Angular subscribes and unsubscribes for you. Zero leak risk.
2. `takeUntil(destroy$)` for imperative subscriptions — a `Subject` | `.next()` + `.complete()` in `ngOnDestroy`.
3. Manual `Subscription.unsubscribe()` only for one-off cases.

In Subsea. The `takeUntil(destroy$)` pattern was my standard in Schedule-Manager. Every component that subscribed imperatively (form `valueChanges`, grid state streams) piped through it:

```
this.filterForm.get('category')!.valueChanges
  .pipe(takeUntil(this.destroy$))
  .subscribe(categoryId => this.loadSubcategories(categoryId));
```

And where I could, I bound streams straight to the template with the `async` pipe so there was nothing to clean up manually:

```
<kendo-dropdownlist [data]="subcategories$ | async"></kendo-dropdownlist>
```

Follow-up: "Anything newer?" In Angular 16+ there's `takeUntilDestroyed()` from `@angular/core/rxjs-interop`, which removes the boilerplate `destroy$` subject — that's the direction I'd move toward.

Q5. How do you restrict / avoid multiple duplicate API calls? *(they've asked this)*

Answer. The classic case is a search box or a filter that fires an API call on every keystroke — you get a flood of requests and race conditions where a slow early response overwrites a fast later one. I fix it with a three-operator combo:

- `debounceTime(ms)` — wait until the user stops typing before calling.
- `distinctUntilChanged()` — skip the call if the value didn't actually change.
- `switchMap()` — **cancel the previous in-flight request** when a new value arrives, so only the latest wins (no race conditions).

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),           // wait for typing to settle
  distinctUntilChanged(),     // ignore no-op changes
  switchMap(term => this.scheduleService.search(term)), // cancel stale request
  takeUntil(this.destroy$)
).subscribe(results => this.rows = results);
```

In Subsea. The grid's filter/search would otherwise hammer the .NET API as the user typed or rapidly changed filters. I used `debounceTime` + `switchMap` so rapid filter changes collapsed into a single, latest request — `switchMap` cancelling the stale grid query was important because otherwise an earlier, slower response could land *after* a newer one and show wrong data in the grid.

Follow-up: "Why switchMap and not mergeMap here?" Because I *want* to abandon the stale request — with `mergeMap` all requests run in parallel and any of them could resolve last and overwrite the grid. `switchMap` guarantees latest-wins. (`concatMap` would queue them, which is worse for search — you'd wait through stale calls.)

Q6. Reactive forms vs template-driven forms — which did you use and why?

Answer.

- **Template-driven** — form logic lives in the HTML with `ngModel`; simple, quick, good for small forms; harder to unit-test and to do dynamic/cross-field logic.
- **Reactive** — form model defined in TypeScript (`FormBuilder` / `FormGroup` / `FormControl`); typed, testable, and it shines for dynamic behavior because I can subscribe to `valueChanges`, add/remove validators at runtime, and drive dependent fields.

I default to **reactive forms** for anything non-trivial.

In Subsea. The **cascading category/subcategory** filter was reactive precisely *because* it's dynamic — selecting a category has to reactively reload the subcategory options. I subscribed to the category control's `valueChanges` and reloaded dependents:

```
this.filterForm = this.fb.group({
  category: [null, Validators.required],
  subcategory: [{ value: null, disabled: true }] // disabled until a category is picked
});

this.filterForm.get('category')!.valueChanges.pipe(
  distinctUntilChanged(),
  switchMap(catId => this.categoryService.getSubcategories(catId)), // cancel stale
  takeUntil(this.destroy$)
).subscribe(subs => {
  this.subcategories = subs;
  const subCtrl = this.filterForm.get('subcategory')!;
  subCtrl.reset();
  subs.length ? subCtrl.enable() : subCtrl.disable(); // enable only when there are options
});
```

That's the reactive-forms sweet spot — a template-driven version of this cascade would have been much messier.

Q7. How did the cascading category/subcategory management work?

Answer. Categories and their subcategories were **data-driven, not hardcoded** — admins could add/edit them, and the UI had to reflect that hierarchy live. I modeled it as a **parent/child relationship** coming from the .NET API, and rendered **cascading selects**: choosing a category filters (and enables) its subcategories.

In Subsea. The key design choices: - On category change I **only reload the dependent subcategories**, not the entire tree — cheaper and snappier. - The subcategory control starts **disabled** and enables when a category with children is chosen. - I used `switchMap` on the category `valueChanges` so if the user changed category quickly, stale subcategory fetches were cancelled. - The whole thing was **reactive-form-driven**, so the selected category/subcategory flowed straight into the grid's server-side filter.

Because it was data-driven, adding a new category was a data change in the backend, not a code change — that was the whole point, and it's what made me pull the selector into a **reusable component** so other parts of the app could drop in the same cascade.

Q8. How does Angular change detection work? Did you use OnPush?

Answer. Angular's default change detection dirty-checks the component tree whenever something *might* have changed — triggered by zone.js patching async APIs (events, timers, XHR).

`ChangeDetectionStrategy.OnPush` narrows that: the component is only re-checked when an `@Input` **reference** changes, an event fires inside it, or an `async`-pipelined observable emits — a real perf win on large trees.

In Subsea (honest). The app ran mostly **default** change detection. Where it mattered for the grid, I leaned on the fact that **server-side data operations** meant each render only dealt with **one page of rows**, not the whole dataset — so change detection had far less to chew on. That's actually a bigger perf lever than OnPush here: I reduced *how much data* is in the view rather than only how often it's checked. Combined with the `async` **pipe** and **Kendo's virtual scrolling**, the grid stayed smooth.

What I'd improve. Heavy reusable grid/list components are good **OnPush** candidates — with the `async` pipe and immutable page updates, that would cut redundant checks further. That's a concrete perf story I can tell honestly.

Q9. Route guards and lazy loading — did you use them?

Answer. **Route guards** (`CanActivate`) gate whether a route can be entered — typically an auth/role check that returns a boolean or `Observable<boolean>`. **Lazy loading** (`loadChildren`) means a feature module isn't bundled into the initial download — it loads on demand when you navigate to it, which shrinks the initial bundle and speeds up first load.

In Subsea. Schedule-Manager was a lazy-loaded feature module — it had its own routing module and shipped as its own chunk, so users who never opened it didn't pay for it on initial load:

```
// app-routing.module.ts
{ path: 'schedule',
  loadChildren: () => import('./schedule-manager/schedule-manager.module')
    .then(m => m.ScheduleManagerModule),
  canActivate: [AuthGuard] }
```

Protected routes went through a `CanActivate` guard that checked the user's auth state before activating the route (and redirected to login otherwise). Structurally, keeping Schedule-Manager as its own lazy module also reinforced the **modularity** work — it kept the module's dependencies from leaking into unrelated areas.

Q10. How do you handle errors and show them to users? (global exception handling)

Answer. I never let raw HTTP errors reach the component. On the Angular side I use two layers: 1. An `HttpInterceptor` that catches failed responses centrally with `catchError` (and handles cross-cutting concerns like auth headers). 2. A global `ErrorHandler` for uncaught client-side errors.

Both funnel into a shared notification service so error UX is consistent, and the **.NET side** returns a **uniform JSON error shape** (not a raw 500 stack trace) via middleware / an exception filter — so the frontend always gets a predictable `{ message }` to show.

In Subsea. Error handling for Schedule-Manager went through a **shared HTTP interceptor** plus **per-call** `catchError` that mapped the backend's error into a user-facing toast:

```
this.scheduleService.saveSchedule(payload).pipe(
  catchError((err: HttpResponse) => {
    this.notify.error(err.error?.message ?? 'Could not save the schedule.');
```

```
    return EMPTY; // swallow so the stream completes cleanly
  }),
  takeUntil(this.destroy$)
).subscribe(() => this.notify.success('Saved.'));
```

So a validation failure on the .NET side came back as clean JSON, the interceptor/ `catchError` picked it up, and the user saw a readable message instead of a broken grid. (The .NET side of this is in `03-backend-dotnet.md`.)

Q11. How did you build reusable components?

Answer. I extract anything that gets copy-pasted into a **configurable shared component** driven by `@Input()` / `@Output()`, so behavior lives in one place — fix a bug once, everyone benefits.

In Subsea. Two big ones: - **A reusable Kendo grid wrapper** — I pulled the repeated grid configuration (columns, server-side data-binding wiring, paging/sort/filter setup) out of individual pages into one component that takes the config and current data page as inputs and emits state changes. Other modules could drop in a consistent, server-side-bound grid instead of re-implementing Kendo setup each time. - **The category/subcategory cascade selector** — packaged as a shared component so any screen needing that hierarchy reused it.

The payoff was consistency and speed: a new grid or a new place needing the category cascade became a **configuration change, not new bespoke code**. This was the core of my modularity/ reusability refactor, and it also tightened the feature-module boundaries so Schedule-Manager didn't leak into unrelated areas.

Q12. How did you manage state?

Answer. Feature-scoped Angular services holding `BehaviorSubject`s — the "service with a subject" pattern. A private `BehaviorSubject` holds the current value, exposed read-only via `asObservable()`, updated only through setter methods. Single source of truth, reactive consumers, no NgRx boilerplate — appropriate for the app's scale.

In Subsea. The selected category/subcategory and grid state lived in feature services with `BehaviorSubject`s. Components read via the exposed observable (with the `async` pipe or `takeUntil()`) and updated through methods — they couldn't push into the subject directly, which kept the state encapsulated. Grid data itself came from the .NET API as Observables and was rendered with server-side state objects.

Rapid-fire recap

Topic	Subsea pattern
Lifecycle hooks	<code>ngOnInit</code> (build form + first load), <code>ngOnDestroy</code> (complete <code>destroy\$</code>), <code>ngOnChanges</code> (reusable child re-render)
Component comms	<code>@Input</code> / <code>@Output</code> on reusable grid; <code>BehaviorSubject</code> service for category state
Observable/Subject/ BehaviorSubject	grid data = Observable; category state = <code>BehaviorSubject</code> (replays current value)
Memory leaks	<code>async</code> pipe first, else <code>takeUntil(destroy\$)</code> completed in <code>ngOnDestroy</code>
Restrict duplicate calls	<code>debounceTime</code> + <code>distinctUntilChanged</code> + <code>switchMap</code> (cancel stale)
Forms	reactive — cascade needs dynamic <code>valueChanges</code> -driven behavior
Cascading dropdowns	category <code>valueChanges</code> → <code>switchMap</code> reload subcategories, enable/disable
Change detection	default; server-side paging keeps view data small; <code>async</code> pipe + virtual scroll
Guards / lazy load	<code>CanActivate</code> on protected routes; Schedule-Manager <code>loadChildren</code> chunk
Error handling	HTTP interceptor + <code>catchError</code> → toast; uniform JSON from .NET
Reusable components	shared Kendo grid wrapper + category cascade selector
State	service + <code>BehaviorSubject</code> , read-only via <code>asObservable()</code>

2. Kendo UI Grid — Deep Dive (THE signature file)

This is your strongest story. You moved Subsea's Kendo Grid from **client-side** to **server-side** data operations and **won a client-vs-server-side debate by demonstrating the lag**. If you nail one thing in the interview, make it this — it shows real engineering judgment, measurement over opinion, and full-stack ownership (Angular *and* .NET).

Stack context: Angular frontend using **Kendo UI for Angular** (the Grid), backend is **.NET (C#) Web API**, data in SQL. The grids displayed large schedule/equipment datasets with sorting, filtering, grouping, and inline editing.

Q1. What is the Kendo Grid and why was it the right choice for Subsea?

Answer. Kendo UI is a commercial component library; the **Grid** is its flagship — a data table with built-in **paging, sorting, filtering, grouping, virtual scrolling, and inline editing**. Subsea needed to show large marine-equipment and schedule datasets with all of those operations, and building that from scratch in a plain `*ngFor` table would have been a huge, buggy effort. Kendo gave us those capabilities out of the box and, crucially, a **first-class pattern for pushing data operations to the server** — which became the centerpiece of my performance work.

In Subsea. The Schedule-Manager grids were the heart of my module — equipment and operation schedules with lots of rows and columns, and users constantly sorting/filtering/grouping them.

Q2. Client-side vs server-side data binding — explain the difference. *(the core question)*

Answer. It's about **where the paging/sorting/filtering happens**:

- **Client-side (local data):** the browser loads the **entire dataset** once, and Kendo does paging/sort/filter **in JavaScript, in memory**. Fine for a few hundred rows. As the dataset grows it falls apart: a big initial payload over the wire, high browser memory, and every sort/filter/group churns the whole array on the main thread → visible lag and jank.
- **Server-side (remote data operations):** the grid sends the **current state** (page number, page size, sort descriptors, filter descriptors) to the backend; the **.NET API translates that into a SQL query** and returns **just the one page** of rows plus the **total count**. The browser only ever holds what's on screen. This scales to arbitrarily large datasets.

	Client-side	Server-side
Data loaded	entire dataset up front	one page at a time
Where sort/filter runs	browser (JS, in memory)	SQL in the .NET API
Payload size	huge, grows with data	small, constant
Browser memory	high	low

	Client-side	Server-side
Scales to large data?	No	Yes
Best for	small, static lists	large/growing datasets (Subsea)

In **Subsea**. The grids started **client-side** and got sluggish as rows grew. I moved them to **server-side** so only one page ever crossed the wire — and that's the change I had to defend in the client-vs-server debate.

Q3. Walk me through the exact server-side pattern you implemented. (show the code)

Answer. Kendo for Angular gives helpers that serialize the grid's `State` (page/sort/filter/group) into a query the .NET side understands, and the .NET Kendo extensions turn a `DataSourceRequest` straight into a paged SQL query. The two sides fit together like a contract.

Angular side — serialize the grid state and call the API:

```
import { toDataSourceRequestString, translateDataSourceResultGroups }
  from '@progress/kendo-data-query';
import { State } from '@progress/kendo-data-query';
import { GridDataResult } from '@progress/kendo-angular-grid';

@Injectable({ providedIn: 'root' })
export class ScheduleGridService {
  constructor(private http: HttpClient) {}

  // state carries { skip, take, sort, filter, group }
  fetch(state: State): Observable<GridDataResult> {
    const query = toDataSourceRequestString(state); // -> querystring the .NET side reads
    return this.http.get<{ data: any[]; total: number }>(`/api/schedule?${query}`)
      .pipe(map(res => ({ data: res.data, total: res.total } as GridDataResult)));
  }
}
```

Component — bind the grid and re-fetch on every state change:

```
export class ScheduleGridComponent implements OnInit {
  public gridState: State = { skip: 0, take: 20, sort: [], filter: undefined };
  public gridData: GridDataResult;

  ngOnInit(): void { this.load(); }

  // fires on paging, sorting, filtering, grouping
  public onStateChange(state: State): void {
    this.gridState = state;
    this.load();
  }

  private load(): void {
    this.gridService.fetch(this.gridState)
  }
}
```

```

        .pipe(takeUntil(this.destroy$))
        .subscribe(result => this.gridData = result);
    }
}

```

```

<kendo-grid [data]="gridData" [pageSize]="gridState.take" [skip]="gridState.skip"
           [sort]="gridState.sort" [filter]="gridState.filter"
           [pageable]="true" [sortable]="true" [filterable]="true" [groupable]="true"
           (dataStateChange)="onStateChange($event)">
  <kendo-grid-column field="assetName" title="Asset"></kendo-grid-column>
  <kendo-grid-column field="scheduledDate" title="Date" filter="date"></kendo-grid-column>
  <kendo-grid-column field="status" title="Status"></kendo-grid-column>
</kendo-grid>

```

.NET side — let Kendo's server extensions do paging/sort/filter in the query:

```

[ApiController]
[Route("api/schedule")]
public class ScheduleController : ControllerBase
{
    private readonly IScheduleService _service;
    public ScheduleController(IScheduleService service) => _service = service;

    // [DataSourceRequest] binds the querystring (page/sort/filter/group) into the request
    object
    [HttpGet]
    public DataSourceResult Get([DataSourceRequest] DataSourceRequest request)
    {
        IQueryable<Schedule> query = _service.QuerySchedules(); // IQueryable — NOT yet
        executed
        return query.ToDataSourceResult(request); // translates to SQL: WHERE + ORDER BY +
        paging
    }
}

```

The key insight: `ToDataSourceResult(request)` is applied to an `IQueryable`, so the paging/sort/filter is composed into the **SQL query** and executed in the database — only the requested page of rows is materialized and returned, wrapped as `DataSourceResult { Data, Total }`. The `Total` drives the grid's pager.

In Subsea. This is exactly the pattern I built for Schedule-Manager. The `State` from the Kendo grid → `toDataSourceRequestString` → `querystring` → `[DataSourceRequest]` on the controller → `ToDataSourceResult` over `IQueryable` → one page of SQL rows back. That round-trip is what kept the grid responsive on large data.

Q4. Why exactly was client-side slow, and how did you *prove* it? (the debate story)

Answer. Client-side was slow for three compounding reasons on a large dataset: 1. **Initial payload** — the API had to serialize and ship *every* row before the grid rendered anything. 2. **Browser memory** — the

whole array sat in memory. 3. **Main-thread work** — every sort/filter/group re-processed the entire array in JavaScript, blocking the UI so the grid felt laggy and unresponsive.

There was a genuine disagreement on the team about whether server-side was worth the effort. Rather than argue in the abstract, I **demonstrated it**: I loaded the grid with a **realistic data volume** and let people see the lag on client-side sort/filter, then showed the server-side version staying snappy on the same data. Seeing the difference ended the debate — we agreed on server-side, and I committed to it fully.

In Subsea. That's my proudest engineering-judgment moment on this project: I didn't win on opinion, I won on **evidence and user impact**. And once the decision was made I owned it end to end — the Angular wiring and the .NET `DataSourceRequest` handling.

The lesson I'll state: "I focus on the user impact and let a measurement settle the argument. I'll advocate hard for the right approach, but once the team decides, I commit — even if it isn't the one I proposed."

Q5. What is `DataSourceRequest` / `DataSourceResult`?

Answer. - `DataSourceRequest` (server side, `[DataSourceRequest]` model binder) — the deserialized grid state: which page (`skip` / `take` ... expressed as page + pageSize), the **sort descriptors**, the **filter descriptors**, and **group descriptors**. It's the request contract the grid sends. - `DataSourceResult` — the response contract: `Data` (the current page of rows) + `Total` (the full unpaged count so the grid can render the pager) + aggregates/groups if requested. - `ToDataSourceResult(request)` — the extension method that takes your `IQueryable` and applies filter → sort → paging as a composed query, executes it, and packages the `DataSourceResult`.

In Subsea. I relied on that contract so I didn't hand-write paging/sorting SQL — the Kendo server extensions translated the grid state into the query, and I returned the `DataSourceResult` straight from the controller.

Q6. Paging, sorting, filtering, grouping — how do they each work server-side?

Answer. With the server-side pattern, all four are just descriptors in the `DataSourceRequest`, and `ToDataSourceResult` maps each to SQL: - **Paging** → `skip` / `take` → SQL `OFFSET/FETCH` (only that page is materialized). - **Sorting** → sort descriptors → SQL `ORDER BY`. - **Filtering** → filter descriptors (per-column, with operators like `contains`, `eq`, date ranges) → SQL `WHERE`. - **Grouping** → group descriptors → grouped aggregation; Kendo can request server grouping and lazy-load group contents.

The grid raises **one** `dataStateChange` event whenever any of these change; I re-send the whole state and get the correct page back — I don't handle each operation separately.

In Subsea. Users sorted by date/status, filtered equipment by category, and grouped operations — all of it resolved in the .NET API against SQL, so the browser never did the heavy lifting.

Q7. What is virtual scrolling and did you use it?

Answer. **Virtual scrolling** renders only the rows currently in the viewport (plus a small buffer) into the DOM instead of all rows, and swaps them as you scroll. It keeps the DOM small even for large pages, which keeps scrolling smooth. With Kendo it pairs naturally with server-side data — the grid requests the next slice as you scroll.

In Subsea. On top of server-side data operations, I used **virtual scrolling** and made sure we **only requested the columns actually displayed**. The combination — server-side paging/sort/filter + virtual scrolling + lean columns — is what kept the grid responsive even on large schedule/equipment datasets.

Q8. How did inline editing / updates work with the grid?

Answer. Kendo's grid supports inline/cell editing that emits edit events; on save I sent the changed row to a .NET `[HttpPut]` / `[HttpPost]` endpoint, and on success re-loaded the current page so the grid reflected server truth (including any server-side recalculation). Keeping the grid bound to server state avoided the client and server drifting apart.

In Subsea. A common production bug class here was **grid/state defects** — filters or edits not persisting correctly after a reload. Because the grid was bound to **server-side state**, the fix was usually making sure the state object (page/sort/filter) was preserved across the edit-and-reload cycle rather than silently reset. (More on that methodology in `05-behavioral-and-stars.md`.)

Q9. Any downsides to server-side, and how did you handle them?

Answer. Honestly, yes: - **More network round-trips** — every sort/filter/page is an API call. I mitigated with `debounceTime` on filter inputs and `switchMap` to cancel stale requests (so rapid filtering doesn't stack calls or cause race conditions — see `01-frontend.md`). - **Backend has to be efficient** — the SQL needs proper **indexing** on sorted/filtered columns, or you just move the slowness to the DB. I made sure the columns we sort/filter on were indexed. - **Slightly more complex** than "load it all" — but that complexity is the price of scaling, and I'd argue for it from the start next time.

In Subsea. One thing I'd standardize earlier: we had a **mix of client- and server-bound grids** across the app. Unifying on the server-side pattern from day one would have prevented the performance issues entirely — that's my honest "what I'd do differently."

Rapid-fire recap

Topic	Subsea answer
Grid used for	large schedule/equipment datasets with sort/filter/group/edit
Client vs server	moved from client-side to server-side data operations
The pattern	Kendo <code>State</code> → <code>toDataSourceRequestString</code> → <code>[DataSourceRequest]</code> → <code>ToDataSourceResult(IQueryable)</code> → <code>DataSourceResult { Data, Total }</code>

Topic	Subsea answer
Why server-side	one page over the wire vs whole dataset in browser memory
How I won the debate	demonstrated the lag on realistic data volume — evidence, not opinion
Paging/sort/filter/group	all become SQL <code>OFFSET/FETCH</code> / <code>ORDER BY</code> / <code>WHERE</code> / grouping
Virtual scrolling	yes — only render viewport rows; plus request only displayed columns
Downsides handled	debounce + <code>switchMap</code> for round-trips; indexed the sort/filter columns
What I'd change	unify all grids on server-side from the start

3. Back-End (.NET / C# Web API) — Interview Q&A tied to Subsea

Honest framing up front: I'm **Angular-emphasis full-stack**. I owned **Schedule-Manager end to end** — the Angular UI *and* the .NET Web API endpoints behind it. So I can tell you exactly how what I built worked: the controllers, the `DataSourceRequest` handling, the data flow, the error shape. Where I'd defer is deep C# runtime/GC internals — I'd bring in a backend specialist for that. Interviewers respect this framing far more than bluffing.

Stack: .NET (C#) Web API, SQL data store, Kendo server-side extensions, containerized with Docker.

Q1. What is a Web API controller and how did you build your endpoints?

Answer. A **Web API controller** is a C# class (inheriting `ControllerBase`, marked `[ApiController]`) whose methods (**actions**) handle HTTP requests. Attributes map HTTP verbs and routes to methods: - `[HttpGet]` — read - `[HttpPost]` — create - `[HttpPut]` — update - `[HttpDelete]` — delete - `[Route("...")]` — the URL template

The action returns data (serialized to JSON) or an `ActionResult` / `ActionResult<T>` so I can control the status code.

In Subsea. I built the Schedule-Manager endpoints — the schedule GET that the Kendo grid calls, plus create/update for schedule entries and the category/subcategory lookups:

```
[ApiController]
[Route("api/schedule")]
public class ScheduleController : ControllerBase
{
    private readonly IScheduleService _service;
    public ScheduleController(IScheduleService service) => _service = service;

    [HttpGet]
    public DataSourceResult Get([DataSourceRequest] DataSourceRequest request)
        => _service.QuerySchedules().ToDataSourceResult(request);

    [HttpPost]
    public ActionResult<ScheduleDto> Create([FromBody] ScheduleDto dto)
    {
        var created = _service.Create(dto);
        return CreatedAtAction(nameof(Get), new { id = created.Id }, created);    // 201
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, [FromBody] ScheduleDto dto)
    {
        _service.Update(id, dto);
        return NoContent();    // 204
    }
}
```

[FromBody] binds the JSON request body to the DTO; the route/verb attributes map the URL and method.

Q2. Explain the `DataSourceRequest` handling on the .NET side.

Answer. This is the server half of my Kendo server-side story (full version in `02-kendo-grid-deep-dive.md`). The `[DataSourceRequest]` attribute is a Kendo model binder that deserializes the grid's state from the querystring — page, sort descriptors, filter descriptors, group descriptors — into a `DataSourceRequest` object. Then `ToDataSourceResult(request)`, called on an `IQueryable`, composes that into a single SQL query (filter → sort → paging) and returns a `DataSourceResult` with the page of `Data` and the total `Total` count.

In Subsea. The whole reason the grid stayed fast on large datasets is that this ran against `IQueryable` so paging/sort/filter executed **in SQL**, not in memory — only one page of rows came back. I built that endpoint; it was the backend counterpart to the frontend grid I also owned.

Q3. Interface vs abstract class in C# — when do you use each? *(they've asked this)*

Answer. Both let you program against an abstraction, but they answer different questions:

	Interface	Abstract class
Represents	a capability / contract ("can-do")	a base type ("is-a")
Implementation	traditionally none (pure contract)*	can have implemented members + abstract ones
Fields/state	no instance fields	can have fields, constructors, state
Inheritance	a class can implement many	a class inherits one
Use when	unrelated types share a capability	related types share a common base + code

*Modern C# allows default interface methods, but I treat interfaces as contracts.

Rule of thumb: use an **interface** to declare *what* something can do when implementers are otherwise unrelated; use an **abstract class** when you have a family of related types that should share real implementation and only override the parts that differ.

In Subsea. My services were defined behind **interfaces** (e.g. `IScheduleService`) — that's a contract, and it's what let me inject them via DI and keep controllers decoupled from the concrete implementation. That interface-based design is also what makes the code testable. I'd reach for an **abstract class** if I had several service types sharing a lot of common logic with a few abstract hooks — a shared base with template behavior.

Q4. Explain `try / catch / finally`. (they've asked this)

Answer. It's structured exception handling: - `try` — wraps code that might throw. - `catch` — runs only if a matching exception is thrown; where I handle/log/translate it. - `finally` — **always runs**, whether or not an exception was thrown (and even if one is rethrown or the method returns) — for **cleanup**: releasing resources, closing connections, resetting state.

```
public ScheduledDto Update(int id, ScheduledDto dto)
{
    try
    {
        return _repository.Update(id, dto);           // risky work
    }
    catch (NotFoundException ex)
    {
        _logger.LogWarning(ex, "Schedule {Id} not found", id);
        throw;                                       // rethrow → global handler maps it to
404
    }
    finally
    {
        _logger.LogInformation("Update attempt finished for {Id}", id); // always runs
    }
}
```

Key point: `finally` is guaranteed cleanup. In modern C# I'd often prefer a `using` statement for `IDisposable` resources (it's `try/finally` with automatic `Dispose()`), but the semantics are the same — deterministic cleanup.

In Subsea. I used `try/catch` around risky operations in the service layer, and let exceptions bubble to a **global handler** (next question) rather than swallowing them — so callers got a clean typed error, not a broken response. `finally / using` was for making sure resources were released regardless of outcome.

Q5. How did you do global exception handling on the backend?

Answer. Rather than `try/catch` in every controller, I centralize it so **one place** turns any unhandled exception into a **uniform JSON error** with the right HTTP status — the frontend then always gets a predictable shape (not a raw 500 stack trace). In .NET this is either **exception-handling middleware** in the pipeline or an **exception filter**.

```
// exception-handling middleware — one place for all unhandled errors
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(RequestDelegate next, ILogger<ExceptionMiddleware> logger)
    { _next = next; _logger = logger; }

    public async Task Invoke(HttpContext ctx)
```

```

    {
        try { await _next(ctx); } // run the rest of the pipeline
        catch (NotFoundException ex) { await Write(ctx, 404, ex.Message); }
        catch (ValidationException ex) { await Write(ctx, 400, ex.Message); }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");
            await Write(ctx, 500, "An unexpected error occurred."); // no stack trace
        }
    }

    private static Task Write(HttpContext ctx, int status, string message)
    {
        ctx.Response.StatusCode = status;
        ctx.Response.ContentType = "application/json";
        return ctx.Response.WriteAsync(JsonSerializer.Serialize(new { message }));
    }
}

```

In Subsea. The Angular side (interceptor + `catchError`) depended on the backend returning a clean `{ message }` with a correct status. This central handler is what produced that — so a validation failure came back as a 400 with a readable message, and the grid/forms showed a toast instead of breaking. The two halves (`01-frontend.md` Q10 and this) are the same story from both ends.

Q6. Dependency injection in .NET — how does it work and did you use it?

Answer. .NET has a **built-in DI container**. You **register** services at startup and the framework **injects** them (usually via constructor) wherever they're needed, so classes depend on **abstractions (interfaces)**, not concrete types — which makes them decoupled and testable. Three lifetimes: - `AddSingleton` — one instance for the whole app. - `AddScoped` — one instance per HTTP request. - `AddTransient` — a new instance every time it's resolved.

```

// Program.cs / Startup
builder.Services.AddScoped<IScheduleService, ScheduleService>();
builder.Services.AddScoped<ICategoryService, CategoryService>();

```

```

// injected via constructor — controller depends on the interface, not the implementation
public ScheduleController(IScheduleService service) => _service = service;

```

In Subsea. My controllers took their services (`IScheduleService`, category service) via **constructor injection**, registered in the app's startup. It's conceptually the same idea as Angular's `providedIn: 'root'` + constructor injection — which is part of why moving between the two sides felt natural to me. I'd typically register data/service classes as **scoped** (per request) so each request got a clean unit of work.

Q7. How were the layers organized?

Answer. Standard layering: - **Controller** — HTTP concerns only: bind the request, call a service, return a status. - **Service** — business logic (interface-defined, DI-injected). - **Repository / data access** — talks to SQL (returns `IQueryable` where I wanted composable queries like the grid's `ToDataSourceResult`). - **DTOs** — the shapes crossing the API boundary, kept separate from internal entities.

In Subsea. Schedule-Manager followed this — thin controllers, an `IScheduleService` for the logic, data access returning `IQueryable` so the Kendo server-side extension could push paging/sort/filter into SQL. Keeping controllers thin also made the endpoints easy to reason about when I was fixing production bugs.

Q8. What are DTOs and why not return the entity directly?

Answer. A **DTO (Data Transfer Object)** is a purpose-built shape for the API boundary. I don't return internal entities directly because: (1) it decouples the API contract from the database schema so I can change one without breaking the other, (2) it avoids over-exposing fields the client shouldn't see, and (3) it prevents serialization issues from entity navigation properties/cycles.

In Subsea. The schedule and category endpoints exchanged DTOs shaped for exactly what the Angular grid and forms needed — which also kept the payloads lean (part of the grid-performance story: only send the columns the UI displays).

Rapid-fire recap

Topic	Subsea answer
Web API controllers	<code>[ApiController]</code> + <code>[HttpGet/Post/Put]</code> + <code>[Route]</code> ; built the Schedule-Manager endpoints
<code>DataSourceRequest</code>	<code>[DataSourceRequest]</code> binds grid state → <code>ToDataSourceResult(IQueryable)</code> → SQL paging
Interface vs abstract	interface = contract/ <code>IScheduleService</code> (DI + testable); abstract = shared base + code
<code>try/catch/finally</code>	try = risky, catch = handle/translate, finally = always-run cleanup (or <code>using</code>)
Global exceptions	middleware/filter → uniform <code>{ message }</code> JSON + correct status (feeds the Angular toast)
DI	register in startup (scoped/singleton/transient), constructor-inject the interface
Layers	thin controller → service (interface) → repo (<code>IQueryable</code>) → DTOs out
Honest depth	owned endpoints end-to-end; defer deep C# runtime/GC internals

4. Docker · Azure · CI/CD — Interview Q&A tied to Subsea

Honest framing: I worked *with* the Docker/Azure setup as a developer — I used the containers to run and integrate my Schedule-Manager work against a realistic backend, and I understand why the setup existed and what it bought us. I wasn't the DevOps owner of the Azure infrastructure, so for deep infra tuning I'd defer to that specialist. But I can explain the workflow clearly because I lived in it daily.

Stack: .NET Web API + Angular frontend, **containerized with Docker**, deployed on **Azure**.

Q1. What is Docker and what problem did it solve on Subsea?

Answer. Docker packages an application and *everything it needs to run* — runtime, dependencies, config — into an **image**, and runs it as an isolated **container**. The image is the same wherever it runs, so you get a **reproducible environment**.

The problem it kills is the classic "**works on my machine.**" Without containers, my laptop, a teammate's laptop, CI, and production can each have subtly different .NET runtime versions, dependencies, or OS libraries — and bugs appear in one place but not another. With Docker, the **same image** runs in all of them, so environment drift disappears.

In Subsea. Docker gave us that reproducible environment — the same image ran on my machine, in CI, and on Azure. That eliminated the whole class of environment-drift bugs, especially around the **.NET runtime and its dependencies**. Concretely, I used the Dockerfile/compose setup to spin up the API and its dependencies locally so I could **integrate my Angular frontend against a realistic backend** instead of mocks — which caught integration issues early.

Q2. Image vs container? And Dockerfile vs compose?

Answer. - **Image** — the immutable, built template (app + dependencies). Like a class. - **Container** — a running instance of an image. Like an object. - **Dockerfile** — the recipe that *builds* an image, step by step (base image, copy code, restore, build, define the start command). - **docker-compose** — defines and runs **multiple containers together** (e.g. the API + its dependencies) with one command and shared networking — great for spinning up a realistic local stack.

In Subsea. The **Dockerfile** built the .NET API image; **compose** let me bring up the API and what it depended on locally as one stack so my frontend had a real backend to talk to.

Q3. Walk me through a Dockerfile for the .NET API.

Answer. A **multi-stage build** — build in an SDK image, then copy just the published output into a small runtime image so the final image is lean and doesn't ship the whole SDK:

```
# --- build stage ---
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore                # restore dependencies (cached layer if csproj unchanged)
COPY . ./
RUN dotnet publish -c Release -o /app/publish

# --- runtime stage ---
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .  # only the published output ships
EXPOSE 8080
ENTRYPOINT ["dotnet", "Subsea.Api.dll"]
```

Why multi-stage matters: the runtime image is small (no SDK/build tools), which means faster pulls/deploys and a smaller attack surface. The layer ordering (restore before copying all source) also means dependency restores are **cached** and only re-run when the project file changes — faster builds.

In Subsea. This is the shape of how the API was containerized — build once, run the same artifact everywhere. The Angular app was similarly built and served as static assets from its own container.

Q4. How did Azure fit in?

Answer. Azure hosted the containers. The Docker images we built were deployed to and run on Azure, so production ran the *same* image that ran locally and in CI. Azure gives managed hosting for containers (services like Azure App Service / Azure Container Apps / AKS), so we got scaling and hosting without managing raw servers.

In Subsea. The flow was: build the container → it runs on Azure in production. The value of Docker + Azure together was **parity** — no "it passed CI but broke in prod" surprises, because prod was running the identical image. Config that differed between environments (connection strings, endpoints) came in through **environment variables**, so the same image was promotable across environments without a rebuild.

Q5. Explain your CI/CD and how you treated the pipeline.

Answer. CI/CD automates build → test → deploy so problems are caught before merge and deploys are repeatable. My working model: **on every PR the pipeline lints, builds, and runs tests**, and I treat a **green pipeline as the gate to deploy** — nothing merges or ships on red. On merge, the pipeline builds the Docker image and deploys it to Azure.

```
PR opened → lint + build + unit tests → (green) → review + merge
merge → build Docker image → push to registry → deploy container to Azure
```

In Subsea. We used Docker for reproducible builds and deployed containers to Azure, so the same image ran locally, in CI, and in prod — which eliminated environment drift. I worked within pipelines that

build and test on every PR, and I treated the green pipeline as the gate before anything went out. That discipline is a big part of why fixes and features shipped safely.

Q6. What does "environment parity" mean and why do you care?

Answer. Environment parity = local, CI, and production being as identical as possible. I care because divergence is where the nastiest bugs live — the ones that only reproduce "over there." Docker delivers parity by running the **same image** everywhere; environment-specific values are injected as **config/env vars**, not baked into the image, so the *code* is identical and only the *configuration* varies.

In Subsea. This was the concrete payoff: because dev/CI/prod ran the same container image, I rarely hit "works on my machine" issues, and when I was fixing a production bug I could reproduce it locally in the same environment shape — which is exactly what my reproduce-first debugging approach depends on (see [05-behavioral-and-stars.md](#)).

Q7. If asked about scaling/resilience on Azure (honest scope)

Answer. At a high level: because the API is **stateless and containerized**, it scales **horizontally** — Azure runs more container instances behind a load balancer under load. Config/secrets come from the environment (Azure's app settings / key vault) rather than the image. That's my working understanding as the developer who built and ran against these containers; the deep infra tuning (autoscale rules, networking, cost) was owned by our DevOps side, and I'd partner with them for that.

In Subsea. I stayed honest about this boundary — I owned my module's code and its containerized run, and understood the deployment model; I didn't own the Azure infrastructure config, and I'd say so rather than overclaim.

Rapid-fire recap

Topic	Subsea answer
Docker's value	reproducible image → killed "works on my machine" (esp. .NET runtime drift)
Image vs container	image = template, container = running instance
Dockerfile	multi-stage: SDK build → lean aspnet runtime image
compose	bring up API + deps locally so my Angular UI had a real backend
Azure	hosts the containers; prod runs the same image as local/CI
CI/CD	PR → lint/build/test → green gates merge → build image → deploy to Azure
Env parity	same image everywhere; env vars for config → fewer "works over there" bugs
Honest scope	owned code + containerized run; deep Azure infra was DevOps' domain

5. Behavioral, Pitch & STAR Stories — Subsea

Everything here is first-person and true to what you built. The interviewer is testing *project understanding* — these are the answers that prove you actually lived this project. Say them out loud until they're natural. Lead with the story, not the tech.

The 60-second pitch (memorize this)

"Subsea is a marine equipment and operations management platform — it's used to track and schedule subsea assets and operations. I worked on it from April to September 2024. The frontend is Angular, the backend is a .NET C# Web API, containerized with Docker and deployed on Azure, and we used Kendo UI for the data grids.

My main ownership was the Schedule-Manager module, which I built front to back — the Angular UI and the .NET endpoints behind it. Inside that I did four things I'm proud of: I moved our Kendo grids from client-side to server-side data operations so large datasets stayed fast; I built dynamic category and subcategory management with cascading dropdowns; I refactored repeated grid and selector code into reusable components; and I regularly fixed production bugs. The team was around five or six people, and I was the full-stack developer with an Angular emphasis who owned Schedule-Manager end to end."

If they want it shorter (15 sec): *"Subsea is a marine asset scheduling platform — Angular front end, .NET Web API back end, Docker on Azure, Kendo grids. I owned the Schedule-Manager module end to end, and my signature win was moving the grids to server-side paging/sorting/filtering so they stayed fast on large data."*

Your role (be precise, don't overclaim)

- **Title/scope:** Full-stack developer, **Angular emphasis**, team of ~5–6.
- **Owned:** the **Schedule-Manager** feature module — Angular UI *and* its .NET Web API endpoints.
- **Also did:** picked up **production defects**, and drove a **modularity/reusability refactor**.
- **Honest boundary:** deep C# runtime internals and deep Azure infrastructure were not my domain — I owned my module's code, both sides, and the containerized run of it.

Architecture walkthrough — what to draw on the whiteboard

Draw this left to right and narrate it:

```
[ Angular SPA ] --REST--> [ .NET C# Web API ] --> [ SQL DB ]
Schedule-Manager          Schedule endpoints          schedules,
- Kendo grids              - [HttpGet] DataSourceRequest equipment,
- cascading dropdowns      - ToDataSourceResult(IQueryable) categories
- reactive forms           - global exception middleware
```

```
|
+--- both containerized with Docker ---+
|
[ Azure hosting ]
(same image: local / CI / prod)
```

Narration: "The Angular SPA makes REST calls to .NET Web API services. My Schedule-Manager grids sent their state — page, sort, filter — to a `[HttpGet]` endpoint that used Kendo's `DataSourceRequest` to page in SQL and return just one page. Everything's containerized with Docker, so the same image runs locally, in CI, and on Azure. The category/subcategory data is a parent/child hierarchy the frontend renders as cascading dropdowns."

STAR Story 1 — The server-side grid performance debate (your best story)

Situation. Subsea's Kendo grids showed large schedule and equipment datasets with sorting, filtering, grouping, and inline editing. They were bound **client-side** — the whole dataset loaded into the browser — and as the data grew, the grid got noticeably sluggish on every sort and filter. There was a disagreement on the team about whether it was worth the effort to change the approach.

Task. I believed we should move data operations **server-side** — do paging, sorting, and filtering in the .NET API rather than in the browser — but I needed to convince the team, not just assert it.

Action. Rather than argue in the abstract, I **demonstrated the problem**: I loaded the grid with a **realistic data volume** and let everyone see the lag on client-side sort/filter, then showed a **server-side version** staying responsive on the same data. Then I built it properly: the Angular grid serialized its state with Kendo's `toDataSourceRequestString`, the .NET controller took a `[DataSourceRequest]` and called `ToDataSourceResult` over an `IQueryable`, so paging/sort/filter composed into a **single SQL query** and only one page of rows crossed the wire. I added **virtual scrolling** and made sure we only requested the columns we displayed.

Result. The grid stayed responsive even on large datasets, the team agreed on server-side, and I owned it end to end — both the Angular wiring and the .NET side. **What I take from it:** I win technical arguments with **evidence and user impact**, not volume — and once the team commits to a decision, I commit fully, even if it's not the one I proposed.

(Full technical detail in [02-kendo-grid-deep-dive.md](#).)

STAR Story 2 — Production bug-fix methodology

Situation. I was one of the people who picked up production defects. A common pattern was **grid/state bugs** — filters or inline edits not persisting correctly after a reload.

Task. Fix them so they *stayed* fixed, without the fix bouncing back.

Action. My approach is **reproduce-first, never guess-and-deploy**: 1. Get the **exact repro steps** and the affected **data shape** from the report. 2. **Reproduce locally** against the same data shape — Docker made this reliable because local matched prod. 3. **Isolate the layer** — use the browser network tab to

decide frontend vs .NET API. 4. **Fix at the correct layer** — for the grid-state bugs, that usually meant preserving the grid's state object (page/sort/filter) across the edit-and-reload cycle instead of letting it reset. 5. **Add a guard/regression test** so the specific failure can't silently come back.

Result. Fixes were durable instead of whack-a-mole. **What I take from it:** methodical reproduction is what stops fixes from bouncing — the discipline of reproducing before touching code is more valuable than being fast.

STAR Story 3 — Dynamic category/subcategory management

Situation. Subsea needed **data-driven** categories and subcategories — admins could add or edit them, and the UI had to reflect that hierarchy live. Hardcoding categories was a non-starter.

Task. Build a UI where choosing a category filters its subcategories, driven entirely by data from the API.

Action. I modeled it as a **parent/child relationship** from the .NET API and built **cascading reactive-form dropdowns**. Selecting a category subscribes to the control's `valueChanges`, uses `switchMap` to reload just the dependent subcategories (cancelling any stale request), and enables the subcategory control only when there are children. I reloaded *only* the dependent dropdown, not the whole tree, to keep it snappy — and I packaged the cascade as a **reusable component** so other screens could reuse it.

Result. Adding a new category became a **data change, not a code change**, and the cascade was reused across the module. **What I take from it:** building things **data/config-driven** makes future "changes" cheap — a lot of would-be code changes became configuration.

(Full technical detail in `01-frontend.md` Q6–Q7.)

STAR Story 4 — Modularity / reusability refactor

Situation. Across Schedule-Manager (and beyond) there was **repeated code** — the same Kendo grid configuration and category-selector setup copy-pasted in multiple places — which meant bugs had to be fixed in several spots.

Task. Reduce duplication and make the module cleaner and more reusable.

Action. I extracted the repeated pieces into **shared, configurable components**: a **reusable grid component** (columns + server-side binding + paging/sort/filter, driven by `@Input` / `@Output`) and the **category/subcategory cascade selector**. I also **tightened the feature-module boundaries** so Schedule-Manager didn't leak dependencies into unrelated areas, and kept it as a **lazy-loaded module**.

Result. New grids and new places needing the category cascade became a **configuration change instead of new bespoke code**, and a bug got fixed once in the shared component rather than everywhere. The module was more maintainable and its boundaries were cleaner.

"What would you do differently?"

Answer. *"I'd standardize the grid data-binding pattern across the whole app from the start. We ended up with a mix of client- and server-bound grids, and if we'd unified on server-side operations from day one we'd have avoided the performance issues entirely rather than fixing them after they showed up. I'd also add more automated integration tests around the .NET endpoints — we leaned on unit tests and manual QA, and stronger integration coverage would have caught grid-state regressions earlier."*

This is a strong answer because it's **honest, specific, and shows judgment** — it's the general version of your best win.

Agile / collaboration

Answer. *"We worked in Agile sprints with Jira for stories and bugs and Git PRs for code review. Because I was building both sides of Schedule-Manager, I coordinated closely on the .NET API contracts — agreeing the request/response shapes early so the Angular and backend work fit together. Standup is where I raised blockers early, and I treated a green CI pipeline as the gate before merging or shipping."*

How do you handle a disagreement on technical approach?

Answer. *"The grid client-vs-server debate is my go-to example. I argued for server-side operations, but instead of just asserting it I demonstrated the lag on realistic data so the trade-off was visible. We aligned on server-side, and I committed to it fully. I focus on evidence and user impact, and once the team decides, I own the decision even if it wasn't originally mine."*

How do you approach building a feature end to end?

Answer. *"Clarify the requirement and acceptance criteria, agree the API contract with the backend early — which for Subsea I often owned both ends of — then sketch the component/service breakdown. I build behind a feature branch with the happy path plus loading/empty/error states, add tests as I go, self-review the diff, open a PR, and demo in sprint review. I like to ship a thin vertical slice first, then iterate."*

Questions to ask the interviewer (shows seniority)

- "How are you handling data-heavy views today — client-side or server-side grid operations?"
 - "What does your CI/CD gate look like before a deploy?"
 - "How is state management handled across the frontend — services, or a store like NgRx?"
 - "Where does the team feel its biggest technical debt is right now?"
-

One-line reminders before you walk in

- **Tie every answer back to Subsea.** The interviewer wants project understanding — give it to them unprompted.
- **Lead with the story, drop to code only if pushed.**
- **Your best card is the server-side grid debate** — measurement beat opinion.
- **Be honest about depth** — Angular-emphasis full-stack who owned a module both sides. That's a strength, not an apology.
- **Speak slower than feels natural.** Nerves make you rush.